PROYECTO DE PROGRAMACIÓN

JUEGO DE LA SERPIENTE

Desarrollo en Python con Pygame

Python · Pygame · Programación Modular

Estudiante: Elías Padilla García
Materia: Lógica de Programación
Docente: Estefanía Vanessa Heredia Jiménez

Contenido

01 **Introducción**
Contexto y objetivos del proyecto

02 **Objetivo del Programa**
Funcionalidades principales

03 **Estructura del Proyecto**
Organización modular del código

04 **Funcionamiento General**
Ciclo de vida del videojuego

05 **Estados del Juego**
Máquina de estados implementada

06 **Movimiento de la Serpiente**
Sistema de control y velocidad

07 **Sistema de Crecimiento**
Mecánica de aumento de tamaño

08 **Comida y Colisiones**
Generación y detección

09 **Conceptos Aplicados**
Técnicas de programación utilizadas

10 **Conclusión**
Reflexiones y aprendizajes

SECCIÓN 01

Introducción

Descripción del Proyecto

Este proyecto consiste en el desarrollo del clásico juego de la serpiente utilizando el lenguaje de programación Python y la librería Pygame. Representa una aplicación práctica de conceptos fundamentales de programación en un entorno de desarrollo de videojuegos.

Objetivo Principal

Aplicar y consolidar conocimientos de programación mediante la creación de un videojuego funcional, integrando múltiples conceptos teóricos en un proyecto práctico y completo.

Valor Educativo

El proyecto permite comprender la estructura básica de un videojuego, desde la inicialización hasta la gestión de estados, pasando por el manejo de eventos y la lógica de juego.

Tecnologías Utilizadas

Python
Lenguaje principal

Pygame
Librería gráfica

Contexto

"El juego de la serpiente es un clásico de la programación que desafía al desarrollador a implementar lógica de movimiento, detección de colisiones y gestión de estados en tiempo real."

SECCIÓN 02

Objetivo del Programa

Meta Principal

Desarrollar un **videojuego funcional** que permita al usuario controlar una serpiente en tiempo real, interactuar con elementos del juego y experimentar una mecánica completa de crecimiento y puntuación.

Control

El usuario debe poder controlar la serpiente mediante entrada de teclado (W, A, S, D) de manera fluida y responsiva.

- Movimiento en 4 direcciones

Alimentación

La serpiente debe poder comer comida generada aleatoriamente en el mapa, lo que incrementa su tamaño y puntuación.

- Crecimiento progresivo

Condiciones de Fin

El juego debe finalizar cuando la serpiente colisione con los bordes de la ventana o con su propio cuerpo.

- Detección de colisiones

Sistema de
Puntuación

Múltiples
Partidas

SECCIÓN 03

Estructura del Proyecto

Programación Modular

El programa fue dividido en **dos archivos principales** siguiendo el concepto de programación modular, facilitando la organización, mantenimiento y comprensión del código.

main.py
Archivo Principal

Lógica Principal
Contiene el núcleo del juego

Bucle Principal
while True del juego

Gestión de Estados
Control de flujo del programa

funciones.py
Archivo Auxiliar

Dibujar Elementos
Renderizado gráfico

Generar Comida
Posiciones aleatorias

Mostrar Texto
Puntuación y mensajes

- Separación de responsabilidades
- Código reutilizable
- Fácil mantenimiento

SECCIÓN 04

Funcionamiento General

Estructura Clásica de Videojuego

El juego sigue el **patrón estándar** de desarrollo de videojuegos, implementando un ciclo de vida completo que gestiona desde la inicialización hasta la renderización en pantalla.

1

Inicialización

Carga de librerías (pygame, random) y definición de variables globales como dimensiones de pantalla, colores y velocidad inicial.

2

Creación de Ventana

Configuración de la superficie de dibujo con dimensiones específicas y título de la ventana.

3

Bucle Principal

Ejecución de while True que mantiene el juego activo, procesando eventos y actualizando el estado continuamente.

Eventos
Captura de teclas presionadas

Posiciones
Actualización de coordenadas

Dibujo
Renderizado de elementos

FPS
Control de velocidad

SECCIÓN 05

Estados del Juego

Máquina de Estados

Se implementó una variable llamada 'estado' que permite controlar el flujo del programa de manera eficiente, gestionando diferentes fases del juego.

menu
Estado Inicial

Función
Muestra la pantalla inicial del juego

Contenido
Título, instrucciones y botón de inicio

Transición
Al presionar tecla → 'jugando'

jugando
Estado Activo

Función
Activa toda la lógica del juego

Procesos
Movimiento, colisiones, puntuación

Transición
Al colisionar → 'gameover'

gameover
Estado Final

Función
Muestra el resultado final

Contenido
Puntuación final y opciones

Transición
Al presionar tecla → 'menu'

Múltiples partidas sin cerrar | Flujo de estados claro | Organización modular

SECCIÓN 06

Movimiento de la Serpiente

Sistema de Control

La serpiente se mueve utilizando **variables de velocidad** en los ejes X y Y, permitiendo un control fluido en cuatro direcciones mediante entrada de teclado.

Controles

W

A S D

- **W** Mover arriba
- **S** Mover abajo
- **A** Mover izquierda
- **D** Mover derecha

Variables de Velocidad

velocidad_x
Controla movimiento horizontal

velocidad_y
Controla movimiento vertical

Actualización

En cada ciclo del juego:

```
x += velocidad_x
y += velocidad_y
```

Respuesta inmediata | Sin movimiento inverso | Movimiento por cuadrícula

SECCIÓN 07

Sistema de Crecimiento

Estructura de Datos

La serpiente se almacena en una **lista** donde cada elemento representa un segmento del cuerpo. Esta estructura permite un crecimiento dinámico y eficiente.

Representación

```
serpiente = [
    H  (x, y) ← Cabeza
    1  (x, y)
    2  (x, y)
    3  (x, y) ← Cola
]
```

Cada elemento es una tupla (x, y) con coordenadas

Movimiento Normal

1. **Agregar nueva cabeza**
   Insertar al inicio de la lista
2. **Eliminar cola**
   Remover último elemento

Al Comer

1. **Agregar nueva cabeza**
   Mismo proceso
2. **No eliminar cola**
   ¡Serpiente crece!

SECCIÓN 08

Comida y Colisiones

Generación de Comida

Sistema de aparición

Librería Random

Se utiliza random.randint() para generar posiciones aleatorias en el mapa.

Alineación de Cuadrícula

La comida aparece en posiciones del tamaño del cuadro (20 pixeles) para alinearse perfectamente con la serpiente.

Regeneración

Al ser comida, aparece inmediatamente en una nueva ubicación aleatoria.

Detección de Colisiones

Condiciones de game over

1 Colisión con Bordes

Si la posición de la serpiente supera los límites de la ventana:

```
estado = 'gameover'
```

2 Colisión con Cuerpo

Si la cabeza coincide con cualquier segmento del cuerpo:

```
estado = 'gameover'
```

Posiciones Válidas

Dentro de los límites de la ventana

Verificación

En cada frame del juego

Finalización

Transición inmediata a gameover

SECCIÓN 09

Conceptos Aplicados

Fundamentos de Programación

Durante el desarrollo del proyecto se aplicaron múltiples **conceptos fundamentales** de programación, integrando teoría y práctica en un proyecto completo.

Programación Modular

División del código en archivos separados con responsabilidades específicas.

✓ main.py + funciones.py

Uso de Funciones

Dividir el programa en pequeñas partes que hacen una tarea específica

✓ Dibujar, generar, mostrar

Listas en Python

Almacenamiento dinámico de la serpiente como colección de segmentos.

✓ serpiente = [(x,y), ...]

Condicionales

Toma de decisiones basada en condiciones usando if/elif/else.

✓ if/elif/else

Bucles Infinitos

Mantenimiento del juego activo mediante while True.

✓ while True:

Manejo de Eventos

Captura y respuesta a eventos del teclado con Pygame.

✓ pygame.event

Control de FPS

Regulación de la velocidad del juego mediante reloj de Pygame.

✓ clock.tick(10)

Números Aleatorios

Generación de posiciones aleatorias para la comida.

✓ random.randint()

SECCIÓN 10

Conclusión

Aprendizaje

Este proyecto permitió **comprender la estructura básica de un videojuego** y reforzar conceptos fundamentales de programación de manera práctica y aplicada.

Organización

La **división del código en múltiples archivos** facilitó su organización y comprensión, demostrando los beneficios de la programación modular.

Logros del Proyecto

- Sistema de menú completo
- Múltiples partidas
- Sistema de puntuación
- Detección de colisiones

Python | Pygame | Programación | Desarrollo